

Especificação Técnica v2 — Plataforma de Bicos

Contexto para desenvolvimento com Claude Code. 18/08/2026. Substitui a v1.
Complementa o *MODELO-NEGOCIO-v2.md*. Em conflito, o modelo de negócio prevalece.

Mudanças em relação à v1: arquitetura separada em três contêineres (era Next.js full-stack), self-hosted em VPS Hostinger com Docker Compose por SSH (era Vercel), storage em MinIO (era R2), monorepo com pacote compartilhado de tipos, verificação por WhatsApp com custo zero, infraestrutura **R\$ 0**.

1. O que estamos construindo

Classificado de trabalho extra. A **empresa** publica vagas de bico e paga assinatura mensal. O **trabalhador** se cadastra de graça, recebe notificação das vagas da função e região dele, e se candidata. As duas partes se acertam fora da plataforma.

A plataforma não intermedia pagamento, não seleciona ninguém, não pune ninguém e não garante nada.

O valor central não é "ter um site" — é **roteamento**: a vaga certa chegar na pessoa certa em minutos, coisa que um grupo de WhatsApp com 3.000 pessoas não faz.

2. Princípios inegociáveis

Jurídicos antes de técnicos. Violá-los quebra o modelo de defesa do negócio.

Regra	Como aparece no código
Trabalhador nunca paga	Não existe rota, tabela ou fluxo de cobrança ligado a <i>Worker</i>
Não tocamos no dinheiro do bico	<i>payAmount</i> é informativo. Nenhuma integração de split ou custódia
Não punimos trabalhador	Não existe <i>banWorker</i> , <i>autoDisable</i> , <i>blockByAbsence</i> . Só o próprio usuário desativa a conta
Não garantimos nada	Copy proibida em toda a UI (§2.1)

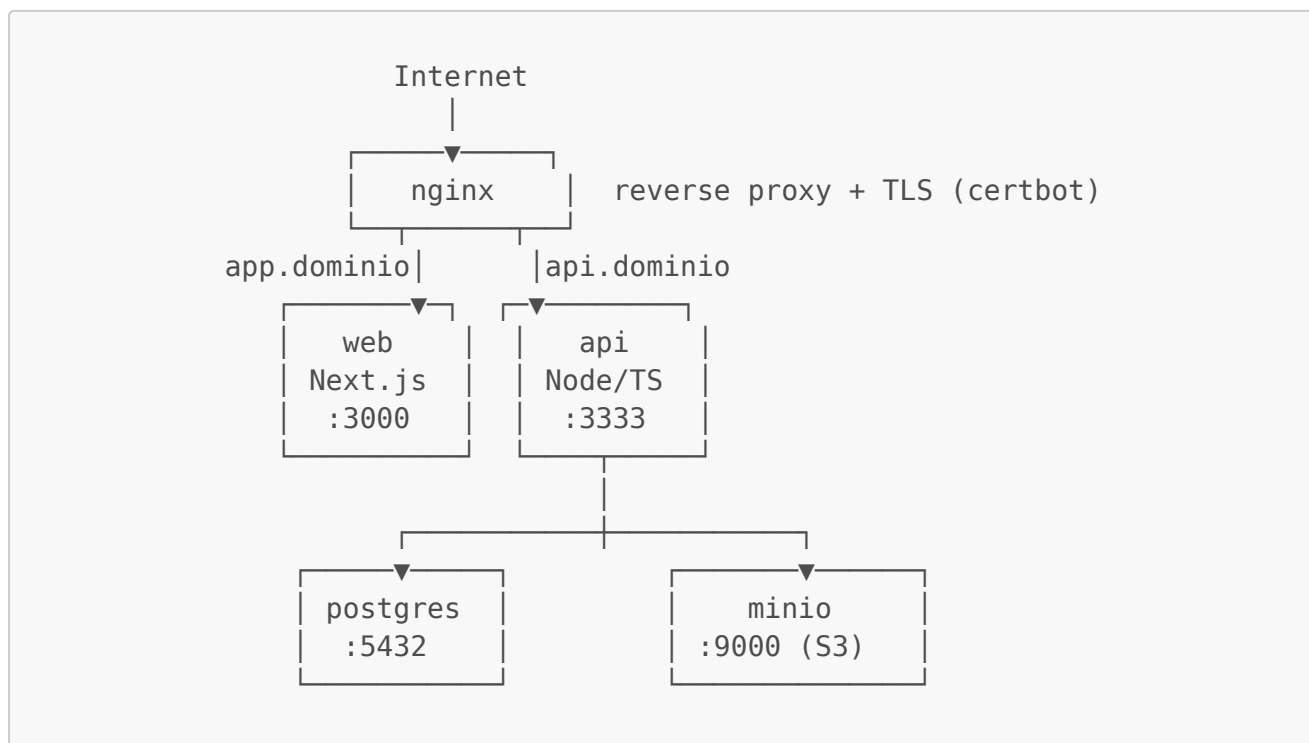
2.1 Vocabulário obrigatório

Nunca escrever	Escrever
verificado, aprovado, confiável	identificado
profissional de confiança	perfil completo
garantimos, asseguramos	histórico informado pelas empresas

O texto da interface é parte da defesa jurídica. Toda copy nova passa por esta tabela.

3. Arquitetura

Três contêineres independentes, mais infraestrutura de apoio. Tudo no VPS próprio.



Rede Docker interna. **Só o nginx expõe porta pública.** Postgres e MinIO nunca ficam acessíveis de fora.

3.1 Por que backend separado (e não Next.js full-stack)

Na v1 eu havia recomendado tudo no Next. Com a informação de que virá um app nativo depois, a separação passa a ser a escolha certa: a API vai servir web e app com o mesmo contrato, e o Next fica sendo só mais um cliente.

O preço dessa escolha, que precisa ser pago desde o começo:

- **Tipos duplicam** se não houver disciplina → resolvido pelo [packages/shared](#) (\$5)
- **CORS e autenticação entre origens** → configuração explícita, tokens em header, não em cookie de sessão
- **Dois pipelines de build e deploy** → resolvido pelo docker-compose único
- **Mais uma coisa para quebrar** → healthcheck em ambos

Se em algum momento o app nativo sair do plano, consolidar em um contêiner só é uma simplificação legítima.

4. Stack e custo

Camada	Escolha	Custo
Proxy/TLS	nginx (contêiner) + certbot para Let's Encrypt	R\$ 0

Camada	Escolha	Custo
Front	Next.js 16.3 App Router, output standalone	R\$ 0
UI	Tailwind + shadcn/ui	R\$ 0
Formulários	react-hook-form + zod	R\$ 0
Estado servidor	TanStack Query	R\$ 0
API	Node 22 + TypeScript + Fastify	R\$ 0
ORM	Prisma	R\$ 0
Banco	PostgreSQL 17 (contêiner, criado na mão)	R\$ 0
Storage	MinIO (contêiner, API compatível com S3)	R\$ 0
Push	Web Push / VAPID + lib web-push	R\$ 0
E-mail	Resend (plano gratuito)	R\$ 0
Orquestração	Docker Compose por SSH no VPS Hostinger	R\$ 0
Verificação de telefone	WhatsApp Cloud API — conversas de serviço	R\$ 0
Domínio	subdomínio de domínio existente até aprovação do sócio	R\$ 0

Total de infraestrutura: R\$ 0. O primeiro custo real só aparece no domínio próprio e, mais tarde, no gateway de assinatura (que só cobra sobre receita).

O Web Push ser gratuito não é detalhe: a funcionalidade mais importante do produto não tem custo por mensagem. É o que torna o modelo viável a custo zero.

4.1 Restrições do público — dirigem decisões técnicas

Trabalhador em **Android de entrada, 4G, dados limitados**. Requisito, não detalhe:

- Bundle JS nas rotas públicas: **< 150 KB** comprimido
- **next/image** sempre, AVIF/WebP
- Vídeo comprimido **no cliente** antes do upload
- Server Components por padrão; **"use client"** só onde há interação
- Meta de LCP em 4G simulado: **< 2,5s**
- Testar em viewport de 360px

5. Monorepo

pnpm workspaces. **O pacote compartilhado é o que impede o backend separado de virar dívida.**

```
extra/
  apps/
    web/                                # Next.js → contêiner "web"
```

```
api/                                # Fastify → contêiner "api"
packages/
  shared/
    src/
      types/                        # contratos de dados (§7)
      schemas/                     # zod – validação usada pelos DOIS lados
      constants/                   # JobRole, palavras bloqueadas, etc.
infra/
  docker-compose.yml
  docker-compose.dev.yml
  nginx.conf
  backup.sh
MODELO-NEGOCIO-v2.md
ESPECIFICACAO-TECNICA-v2.md
CLAUDE.md
```

Regra: **apps/web** e **apps/api** nunca declaram um tipo de domínio localmente. Se um tipo é do domínio, mora em **packages/shared**. O mesmo schema zod valida o formulário no front e o corpo da requisição na API.

6. Abordagem: outside-in com contrato de dados

Telas primeiro, banco por último — mas com contratos definidos antes, para o backend não nascer torto.

Fase 1 — Contratos. Tipos e schemas zod em **packages/shared**. Nenhuma tela antes disso.

Fase 2 — Mock. **apps/web/src/lib/api/** com as assinaturas definitivas, retornando fixtures com latência simulada. Controlado por **NEXT_PUBLIC_API_MODE=mock**.

Fase 3 — Front completo. Todas as telas do MVP navegáveis contra o mock. Vendável sem uma linha de backend.

Fase 4 — API real. Fastify cumprindo os contratos. Troca-se **NEXT_PUBLIC_API_MODE=live**. O front não muda.

Dívida registrada na Fase 3: em modo mock todas as rotas renderizam sob demanda (**f**), porque o store vive no **localStorage** do navegador. Ao ligar o modo live, **/vagas/[slug]** precisa voltar a ser estática ou **ISR** — é a página que o Google indexa, e indexação é a única aquisição gratuita do projeto. Conferir no **next build** que ela aparece como **o** ou **ISR**, não como **f**.

Fase 5 — Banco. Schema Prisma derivado dos tipos já validados na prática. Migrações rodadas na mão no contêiner.

Regra de ouro: **nenhum componente importa de **src/mocks/** diretamente**. Tudo por **src/lib/api/**. Respeitada essa regra, a Fase 4 é troca de implementação, não reescrita. Vale um **eslint-plugin-import** com **no-restricted-imports** para garantir.

7. Contratos de dados

Em `packages/shared/src/types/`. Escrever antes de qualquer tela.

```
// worker.ts
export type WorkerStatus = 'incomplete' | 'complete' |
'self_deactivated'

export interface Worker {
  id: string
  fullName: string
  phone: string // E.164
  phoneVerifiedAt: string | null // verificado via WhatsApp – ver §11
  cpf: string // armazenado; NUNCA em resposta
  pública
  birthDate: string // bloquear < 18 anos (ECA Digital)
  city: string
  neighborhood: string
  roles: JobRole[]
  experience: string
  availability: Availability[]
  documentSelfieKey: string | null // chave no MinIO, bucket privado
  introVideoKey: string | null // 30s, bucket público
  references: WorkerReference[]
  status: WorkerStatus
  attendance: AttendanceSummary
  createdAt: string
}

export interface WorkerReference { name: string; phone: string;
relationship: string }

export interface Availability {
  weekday: 0|1|2|3|4|5|6
  period: 'morning' | 'afternoon' | 'night'
}

// 0 que a empresa enxerga. Sem CPF, sem data de nascimento.
export interface WorkerPublicProfile {
  id: string
  firstName: string
  lastNameInitial: string
  neighborhood: string
  roles: JobRole[]
  experience: string
  introVideoUrl: string | null
  hasCompleteProfile: boolean
  attendance: AttendanceSummary
  memberSince: string
}
```

```
// attendance.ts
export interface AttendanceSummary {
  present: number
  absent: number
  distinctCompanies: number
  // 'not_selected' NÃO entra aqui e NUNCA é exibido ao público. Ver §16.7.
  // NUNCA existe rating, stars, score ou comment.
}

// 'not_selected' é NEUTRO: não entra em present nem em absent. Ver §16.7.
export type AttendanceStatus =
  | 'pending' // aguardando a empresa marcar
  | 'not_selected' // não foi chamado – neutro, não conta como nada
  | 'present' // foi chamado e compareceu
  | 'absent' // foi chamado e NÃO compareceu – só isto é falta
  | 'disputed' // contestado pelo trabalhador

export interface AttendanceRecord {
  id: string
  workerId: string
  companyId: string
  jobPostId: string
  applicationId: string
  status: AttendanceStatus
  markedAt: string | null // null enquanto 'pending'
  disputedAt: string | null // contestação em até 7 dias
  expiresAt: string // markedAt + 12 meses
  // Sem campo de texto livre. É a regra que evita ação por dano moral.
}
```

```
// job.ts
export type JobRole =
  | 'garcom' | 'cozinheiro' | 'auxiliar_cozinha' | 'auxiliar_limpeza'
  | 'diarista' | 'barman' | 'seguranca' | 'recepcionista'
  | 'montagem_evento' | 'motorista' | 'outro'

export type JobStatus = 'open' | 'filled' | 'expired' | 'cancelled'

export interface JobPost {
  id: string
  slug: string
  companyId: string
  role: JobRole
  title: string
  description: string
  date: string
  startTime: string
}
```

```

endTime: string
payAmount: number // INFORMATIVO. Não processamos este valor.
payNote: string | null
address: string
neighborhood: string
city: string
requirements: string | null // uniforme etc – exigência DA EMPRESA
vacancies: number
applicationsCount: number // exibido no card; substitui o contato
maxApplications: number // vacancies * 3 – ver §16.5
status: JobStatus
isHighlighted: boolean
publishedAt: string
expiresAt: string
}

// Nunca faz parte do payload público. Só é servido após candidatura
ativa.
export interface JobPostContact {
  jobPostId: string
  contactPhone: string
}

```

```

// application.ts
export type ApplicationStatus = 'applied' | 'confirmed' | 'withdrawn' |
'no_response'

export interface Application {
  id: string
  shortCode: string // 4 caracteres, ex "A7K2" – vai na
mensagem (§16.5)
  jobPostId: string
  workerId: string
  status: ApplicationStatus
  appliedAt: string
  contactedAt: string | null // quando tocou em "Falar no WhatsApp"
  confirmedAt: string | null // confirmação de véspera
}

```

```

// company.ts
export type SubscriptionStatus =
  | 'trialing' | 'active' | 'past_due' | 'suspended' | 'cancelled'

export interface Company {
  id: string
  cnpj: string
  legalName: string
  tradeName: string
}

```

```

responsibleName: string
phone: string
email: string
city: string
subscriptionStatus: SubscriptionStatus
subscriptionEndsAt: string | null
createdAt: string
}

```

```

// api.ts
export interface Paginated<T> {
  items: T[]; total: number; page: number; pageSize: number
}

export type ApiResult<T> =
  | { ok: true; data: T }
  | { ok: false; error: { code: string; message: string; field?: string } }
}

```

8. Contrato HTTP

Prefixo **/v1**. JSON. Autenticação por **Authorization: Bearer <jwt>**.

```

POST    /v1/auth/request-code      { phone }
POST    /v1/auth/verify-code      { phone, code } → { token, user }
GET     /v1/auth/me

GET     /v1/jobs                  ?
role=&city=&neighborhood=&date=&page=
GET     /v1/jobs/:slug
POST    /v1/jobs                  [empresa]
PATCH  /v1/jobs/:id/close        [empresa]

POST    /v1/jobs/:id/applications [trabalhador] 409 se atingiu
maxApplications
GET     /v1/me/applications        [trabalhador]
GET     /v1/jobs/:id/contact      [trabalhador] 403 sem candidatura
ativa
POST    /v1/applications/:id/contacted [trabalhador] registra contactedAt
POST    /v1/applications/:id/confirm [trabalhador] confirmação de
véspera

GET     /v1/jobs/:id/applicants   [empresa] → WorkerPublicProfile[]
POST    /v1/jobs/:id/attendance   [empresa] { workerId, status }
POST    /v1/attendance/:id/dispute [trabalhador]

POST    /v1/workers               cadastro (multi-etapa, PATCH

```

```

parcial)
PATCH /v1/workers/me
POST /v1/uploads/presign { kind: 'selfie'|'video' } → URL do
MinIO

POST /v1/push/subscribe { subscription }
POST /v1/reports denúncia de vaga ou perfil

```

Erro sempre no formato `ApiResult`. HTTP status coerente (400 validação, 401 sem token, 403 sem permissão, 409 conflito).

8.1 Camada de acesso no front

Mesmas assinaturas em mock e em real. O componente não sabe qual está rodando.

```

// apps/web/src/lib/api/jobs.ts
export async function listJobs(filters: JobFilters):
Promise<Paginated<JobPost>>
export async function getJobBySlug(slug: string): Promise<JobPost |
null>
export async function createJob(input: CreateJobInput):
Promise<ApiResult<JobPost>>

```

O mock simula 300–800ms de latência e falha em ~5% das chamadas — força os estados de carregamento e erro a existirem desde o início, em vez de virarem dívida.

9. Docker

9.1 Compose de produção (`infra/docker-compose.yml`)

```

services:
  nginx:
    image: nginx:alpine
    restart: unless-stopped
    ports: ["80:80", "443:443"]
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ./certbot/conf:/etc/letsencrypt:ro
      - ./certbot/www:/var/www/certbot:ro
    networks: [edge]
    depends_on: [web, api]

  certbot:
    image: certbot/certbot
    restart: unless-stopped
    volumes:
      - ./certbot/conf:/etc/letsencrypt

```

```
- ./certbot/www:/var/www/certbot
entrypoint: >
  /bin/sh -c 'trap exit TERM; while ;; do
  certbot renew --webroot -w /var/www/certbot --quiet;
  sleep 12h & wait $$!}; done;'
```

web:

```
build: { context: .., dockerfile: apps/web/Dockerfile }
restart: unless-stopped
environment:
  NEXT_PUBLIC_API_URL: https://api.SEU-SUBDOMINIO
  NEXT_PUBLIC_API_MODE: live
  NEXT_PUBLIC_VAPID_PUBLIC_KEY: ${VAPID_PUBLIC_KEY}
networks: [edge]
depends_on: [api]
```

api:

```
build: { context: .., dockerfile: apps/api/Dockerfile }
restart: unless-stopped
environment:
  DATABASE_URL:
postgresql://extra:${DB_PASSWORD}@postgres:5432/extra
  JWT_SECRET: ${JWT_SECRET}
  VAPID_PUBLIC_KEY: ${VAPID_PUBLIC_KEY}
  VAPID_PRIVATE_KEY: ${VAPID_PRIVATE_KEY}
  S3_ENDPOINT: http://minio:9000
  S3_ACCESS_KEY: ${MINIO_ROOT_USER}
  S3_SECRET_KEY: ${MINIO_ROOT_PASSWORD}
  RESEND_API_KEY: ${RESEND_API_KEY}
  CORS_ORIGIN: https://app.SEU-SUBDOMINIO
networks: [edge, internal]
depends_on: [postgres, minio]
```

postgres:

```
image: postgres:17-alpine
restart: unless-stopped
environment:
  POSTGRES_USER: extra
  POSTGRES_PASSWORD: ${DB_PASSWORD}
  POSTGRES_DB: extra
volumes: [pgdata:/var/lib/postgresql/data]
networks: [internal] # sem porta publicada
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U extra"]
  interval: 10s
```

minio:

```
image: minio/minio
restart: unless-stopped
command: server /data --console-address ":9001"
environment:
  MINIO_ROOT_USER: ${MINIO_ROOT_USER}
  MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
```

```
volumes: [miniodata:/data]
networks: [internal, edge]
```

```
volumes: { pgdata: {}, miniodata: {} }
networks: { edge: {}, internal: { internal: true } }
```

9.2 nginx (infra/nginx.conf)

Um **server** por subdomínio. Modelo do bloco do app — repetir para **api** e **cdn** trocando o **proxy_pass**:

```
server {
    listen 80;
    server_name app.SEU-SUBDOMINIO;
    location /.well-known/acme-challenge/ { root /var/www/certbot; }
    location / { return 301 https://$host$request_uri; }
}

server {
    listen 443 ssl;
    server_name app.SEU-SUBDOMINIO;

    ssl_certificate      /etc/letsencrypt/live/app.SEU-
SUBDOMINIO/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/app.SEU-
SUBDOMINIO/privkey.pem;

    client_max_body_size 10M;    # upload de vídeo de 30s

    location / {
        proxy_pass http://web:3000;
        proxy_set_header Host                $host;
        proxy_set_header X-Real-IP           $remote_addr;
        proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto  $scheme;
    }
}
```

Emissão do certificado, uma vez por subdomínio, com o nginx já no ar:

```
docker compose run --rm certbot certonly --webroot -w /var/www/certbot \
-d app.SEU-SUBDOMINIO -d api.SEU-SUBDOMINIO -d cdn.SEU-SUBDOMINIO
docker compose exec nginx nginx -s reload
```

O contêiner **certbot** renova sozinho a cada 12h.

Alternativa: o Caddy faz proxy e TLS com quatro linhas de configuração e sem certbot. Só vale trocar se o nginx começar a incomodar — ferramenta conhecida ganha de ferramenta elegante.

9.3 Dockerfiles

- **web:** multi-stage, `output: 'standalone'` no `next.config.js`, `node:22-alpine`, usuário não-root
- **api:** multi-stage, `pnpm deploy --filter api` na build, `prisma generate` antes do bundle
- Ambos com `HEALTHCHECK`

9.4 Desenvolvimento local

`docker-compose.dev.yml` sobe **só postgres e minio**. Web e API rodam no host com hot reload, no WSL. Sem rebuild de imagem a cada alteração.

9.5 Backup — não pular esta parte

Banco em contêiner sem backup é perda de dados marcada para acontecer. `infra/backup.sh` no cron do host:

```
docker exec extra-postgres pg_dump -U extra extra | gzip >
/backups/extra-$(date +%F).sql.gz
find /backups -name 'extra-*.sql.gz' -mtime +14 -delete
```

Testar a restauração uma vez. Backup nunca testado não é backup.

9.6 Deploy no VPS Hostinger

Fluxo padrão: desenvolve no WSL, sobe pro Git, faz pull e deploy no servidor. Sem painel, sem registry, sem CI.

PROF

```
ssh usuario@vps
cd /opt/extra && git pull
docker compose -f infra/docker-compose.yml up -d --build
docker compose logs -f api
```

Um script `deploy.sh` no servidor resolve em um comando.

Único ponto de atenção: o build do Next.js é bem mais pesado que o de uma API Node comum, e num VPS pequeno pode estourar memória. Prevenção, uma vez só:

```
fallocate -l 2G /swapfile && chmod 600 /swapfile
mkswap /swapfile && swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab
```

Com swap, o build passa — só demora alguns minutos. Se um dia incomodar, a saída é build no WSL e `docker save | ssh vps 'docker load'`, sem mudar nada no fluxo de Git.

DNS: apontar `app`, `api` e `cdn` do subdomínio para o IP do VPS. Depois disso, emitir os certificados uma vez (§9.2).

Migrações não entram no deploy automático. Rodar na mão, com backup feito antes (§10).

10. Migrações e banco

- Schema em `apps/api/prisma/schema.prisma`, derivado dos tipos de `packages/shared`
 - Dev: `prisma migrate dev` contra o Postgres local
 - Produção: `prisma migrate deploy` executado na mão dentro do contêiner da API, com backup feito antes
 - **Sem seed automático em produção.** Dados reais entram pela aplicação
 - Índices desde o início: `job_posts (city, role, status, date)`, `applications (job_post_id, worker_id)` único, `workers (phone)` único, `attendance_records (worker_id, expires_at)`
-

11. Autenticação — verificação por WhatsApp, custo zero

Telefone + código de uso único, confirmado pelo WhatsApp. Sem senha, sem e-mail, sem SMS.

SMS é o canal errado para este público: muito número é WhatsApp mas não recebe SMS, e ninguém quer decorar senha. O WhatsApp é onde essas pessoas já vivem.

11.1 O truque: OTP invertido

Enviar um código pelo WhatsApp custa dinheiro — *authentication template* sai por **US\$ 0,0068** por mensagem no Brasil. Mas **conversa iniciada pelo usuário é gratuita e ilimitada** desde novembro de 2024, e qualquer resposta dentro da janela de 24h também é.

Então a gente inverte o fluxo: em vez de a plataforma mandar o código, **o usuário manda**.

1. Usuário digita o telefone no cadastro
2. Backend gera um código de 6 dígitos e devolve um link `wa.me`
3. Botão grande: **"Confirmar no WhatsApp"** → abre `https://wa.me/55DDDNUMERO?text=Meu%20codigo%20e%20482913`
4. Usuário toca em enviar — é isso, um toque
5. O webhook da Cloud API recebe a mensagem contendo o código **e o número do remetente, validado pelo próprio WhatsApp**
6. Backend casa código + número, grava `phoneVerifiedAt`
7. Responde "Confirmado!" na mesma janela — também gratuito
8. Front detecta por polling curto (2s, limite de 3min) e avança a etapa

11.2 Por que isso é melhor que SMS

- **Custo zero e oficialmente suportado.** Sem biblioteca não-oficial, sem risco de banimento do número
- **Mais forte que SMS:** o número chega assinado pelo próprio WhatsApp. Prova que a pessoa controla aquele WhatsApp, não só que recebeu um SMS
- **Funciona em número que só tem WhatsApp** — exatamente o caso de boa parte do público
- Um toque, sem digitar código, sem sair do app que a pessoa já usa

11.3 O que é preciso montar

- Conta no Meta Business + WhatsApp Business Platform (Cloud API)
- **Um número dedicado.** Ao entrar na Cloud API, esse número deixa de funcionar no app comum do WhatsApp — usar um chip separado, nunca o número pessoal e **jamais** o número ligado aos grupos do sócio
- Webhook HTTPS público → `POST /v1/webhooks/whatsapp` (o nginx já entrega TLS)
- Verificação de token do webhook e checagem da assinatura `X-Hub-Signature-256`
- Contas não verificadas têm limite inicial de contatos únicos por 24h. Para fluxo de entrada isso não incomoda no MVP, mas a verificação do Meta Business deve ser feita antes de escalar

11.4 Bordas a tratar

- Código expira em 10 minutos; máximo 3 tentativas por telefone por hora
- Rate limit por IP na geração do código
- Botão "não consegui confirmar" → segue com `phoneVerifiedAt = null` e um aviso discreto no perfil. **Nunca travar o cadastro por falha de verificação** — perder o usuário é pior que ter um telefone não confirmado
- Desktop: exibir também o QR do `wa.me` para quem estiver no computador

11.5 Efeito colateral bom

A mesma integração vira o canal de recado para quem recusar o push. Fora da janela de 24h isso exige *utility template*, a US\$ 0,0068 por mensagem — centavos de real. Não é custo zero, então fica para depois da primeira receita. O Web Push continua sendo o canal principal, e ele é gratuito.

PROF

11.6 Sessão

JWT de vida longa (30 dias) com refresh silencioso. Pedir login a cada visita mata o produto.

12. PWA e notificações

- `manifest.json, display: standalone`
- Service worker próprio — evitar `next-pwa`, que costuma brigar com o App Router
- Web Push com VAPID e a lib `web-push` na API. Chaves geradas uma vez, guardadas no `.env`
- Prompt de instalação **não** na primeira visita: só depois da primeira candidatura, quando o valor já foi percebido
- **Android/Chrome:** funciona normalmente — é o caso da esmagadora maioria do público
- **iOS/Safari:** push só funciona com o PWA adicionado à tela de início. Detectar iOS e exibir instrução de instalação antes de pedir permissão

- Quem recusar push vê as vagas na listagem; o sócio avisa no grupo. Fallback por WhatsApp entra depois da primeira receita (\$11.5)

13. Mídia no MinIO

- Dois buckets: **docs** (**privado**) e **public** (vídeos)
- Upload direto do navegador via URL pré-assinada gerada pela API — **o arquivo nunca passa pelo contêiner da API**
- Selfie com documento: bucket privado, acesso apenas por URL assinada de curta duração. Nunca é conteúdo público
- Selfie comprimida no cliente: máx. 1200px de largura, ~200 KB
- Vídeo: **MediaRecorder**, teto rígido de 30s e ~5 MB, com barra de progresso
- Política de retenção: ao excluir a conta, remover os objetos do MinIO (obrigação de eliminação da LGPD)

14. Regras de negócio no código

14.1 Filtro de linguagem discriminatória

Schema zod em **packages/shared/src/schemas/job.ts**, usado no formulário e na rota da API — o cliente é contornável. Aplica-se a **title**, **description** e **requirements**.

Bloquear: **moça, moço, rapaz, boa aparência, boa apresentação, até \d+ anos, sexo (masculino|feminino), apenas (homens|mulheres), solteir[oa], sem filhos**, recortes raciais.

Mensagem explicativa, não acusatória: *"Anúncios de vaga não podem exigir sexo, idade ou aparência (art. 373-A da CLT). Reescreva descrevendo a função e os requisitos técnicos."*

Registrar tentativas bloqueadas em log — é prova de diligência.

14.2 Bloqueio de menores

Validação de **birthDate** no schema compartilhado. Sem persistência de cadastro parcial de menor.

14.3 Ausência de poder disciplinar

Não implementar, jamais, mesmo se pedido depois: banimento por faltas, desativação automática, bloqueio por nota, ranking punitivo, ordenação que rebaixa por falta. O histórico é **exibido**; a decisão é da empresa.

14.4 Denúncia e remoção

Rota de denúncia em toda vaga e todo perfil. Fila no painel administrativo. Meta de 48h. Toda ação com data registrada — é o que sustenta a defesa diante de notificação extrajudicial.

15. Telas do MVP

Público: home com vagas abertas · listagem filtrável · detalhe da vaga (SSG/ISR, indexável no Google — aquisição orgânica gratuita) · cadastro do trabalhador (6 etapas) · cadastro da empresa · entrar

Trabalhador: minhas candidaturas (com confirmação de véspera em destaque) · meu perfil com indicador de completude

Empresa: painel · publicar vaga · candidatos da vaga (com botão de WhatsApp) · marcar presença · assinatura

16. Fluxos críticos

16.1 Cadastro do trabalhador (6 etapas, ~8 min). Nome + CPF → telefone → selfie com documento → perfil (funções, experiência, disponibilidade, bairro) → vídeo de 30s → duas referências. Progresso salvo a cada etapa: queda de conexão não pode zerar o esforço. `hasCompleteProfile` só vira `true` com as 6 concluídas.

A fricção é intencional — filtra quem não faria o esforço de acordar às 6h no sábado. **Mas é hipótese, não dogma:** se a conclusão ficar abaixo de 40%, o vídeo é o primeiro item a cair.

16.2 Publicação → notificação → candidatura. Empresa publica → filtro de conteúdo → vaga `open` → job seleciona por `role` + região + disponibilidade → dispara Web Push → trabalhador se candidata → empresa vê a lista e chama no WhatsApp.

O tempo entre publicar e a primeira notificação chegar é a métrica técnica mais importante do produto. Meta: menos de 60 segundos.

16.3 Confirmação de véspera. Job diário às 18h notifica os candidatos das vagas do dia seguinte. Sem confirmação até 18h, a vaga reabre e a empresa é avisada. Não confirmar **não gera falta** — é aviso, não punição.

16.4 Marcação de presença. Após a data, pendência no painel da empresa. Um clique por candidato, entre três opções — não chamei / compareceu / não compareceu (§16.7). Sem texto. O trabalhador é notificado e tem 7 dias para contestar; contestado, sai do perfil público até resolução. Expira em 12 meses.

16.5 Revelação de contato — só a empresa inicia.

O telefone **nunca** aparece em página pública, e **nunca é exibido como texto em lugar nenhum da interface** — só existe atrás de um botão que abre o WhatsApp. Número escrito na tela é número copiado e colado no grupo, e aí o anti-raspagem virou enfeite.

Direção única: a empresa entra em contato com o trabalhador. Nunca o contrário.

Motivo: uma vaga de seis aceita até dezoito candidaturas. Se todos pudessem chamar, o contratante — que é o cliente pagante — receberia dezoito mensagens de desconhecidos por anúncio, e cancelaria a assinatura. Além disso, hoje no grupo de WhatsApp o número dele fica exposto a milhares de pessoas; aqui não aparece para ninguém. Isso é argumento comercial, não só proteção.

Fluxo do trabalhador:

1. Abre a vaga. Vê tudo — função, data, valor, local, exigências. **Nunca vê contato**
2. Toca em **"Quero essa vaga"** → cria a `Application` com `shortCode`
3. A tela mostra: **"Candidatura enviada. Se a empresa escolher você, ela chama no seu WhatsApp."**
 - nunca escrever "a empresa foi notificada": é promessa sobre terceiro
4. Não existe botão de contato para o trabalhador. Em nenhum momento

Fluxo da empresa:

1. Recebe aviso e vê os candidatos no painel, com perfil e histórico
2. Toca em **"Falar no WhatsApp"** no candidato escolhido
3. Abre o `wa.me` com mensagem pronta e grava `contactedAt`

```
Oi João! Aqui é o Buffet Encanto.  
Vi sua candidatura para Garçon para casamento em Cascatinha,  
sábado 22/08 às 17h — R$ 150.  
Código: A7K2  
— via extraqui.com.br
```

O clique é o ato de escolher. `contactedAt` preenchido = a empresa chamou.

Isso alimenta a marcação de presença (§16.7): quem foi contatado é candidato a `present/absent`; quem não foi tende a `not_selected`.

Depois desse primeiro contato o trabalhador passa a ter o número da empresa — e tudo bem: é uma conversa um-a-um, depois da escolha, e não uma exposição em massa como no grupo.

Teto de candidaturas: `maxApplications = vacancies * 3`. Atingido o teto, a vaga para de aceitar e exibe "candidatos suficientes".

PROF

Métrica que isso destrava: candidaturas por vaga versus `contactedAt` preenchidos. Muitas candidaturas e poucos contatos significa que a empresa não está voltando ao painel — e aí o problema é de notificação, não de oferta.

16.6 Exibição do histórico de presença.

Formato único, em qualquer tela onde o trabalhador apareça para a empresa:

```
9 presenças · 1 falta · 5 empresas
```

Regras de exibição:

- número cru. Sem estrela, sem nota, sem porcentagem, sem barra de progresso
- **sem cor que sugira julgamento.** Falta em vermelho é amplificador de juízo, e o produto não julga — informa

- registros contestados não entram na contagem enquanto estiverem contestados
- registros com mais de 12 meses não entram
- ao lado, o `shortCode` da candidatura, para casar com a conversa no WhatsApp

A rampa de entrada — regra que não pode ser esquecida:

Quem ainda não tem histórico **nunca** exibe `0 presenças`. Exibe **"Novo por aqui"**, em tom neutro, ao lado do selo de perfil completo.

Motivo: sistema de reputação sem rampa tranca o novato em definitivo — ele não é chamado porque não tem histórico, e não tem histórico porque não é chamado. `0 presenças`, `0 faltas` lê como ruim, não como neutro, e mata a entrada de gente nova, que é exatamente o lado que precisa crescer.

O selo de **perfil completo** é a reputação substituta de quem chegou agora: não tem histórico, mas gravou o vídeo, deu duas referências e preencheu tudo. É o que a empresa olha enquanto o histórico não existe. Por isso os dois aparecem sempre juntos.

Agendamento (16.3 e expiração de vagas): `node-cron` dentro do contêiner da API. Sem fila externa no MVP — o volume não justifica.

17. Padrões de código

- TypeScript `strict: true`. Zero `any`
 - Server Components por padrão; `"use client"` só onde há interação
 - Schemas zod em `packages/shared`, compartilhados entre formulário e rota
 - Funções de `lib/api/` retornam `ApiResponse<T>` — erro é valor de retorno, não exceção
 - Código em inglês; interface em português do Brasil
 - Datas em ISO 8601 UTC no contrato; formatação para `America/Sao_Paulo` só na exibição
 - Sem barrel files
 - Commits convencionais
 - Segredos só em `.env`, nunca no repositório. `.env.example` versionado
-

PROF

18. Ordem de execução

1. Monorepo pnpm, workspaces, lint, TS
2. **Todos os tipos e schemas de `packages/shared`** — nada de tela antes
3. Fixtures e camada mock
4. Layout base, navegação, PWA manifest
5. **Home + listagem + detalhe da vaga** ← *é o que o sócio leva para vender*
6. Cadastro do trabalhador, 6 etapas
7. Login (mockado)
8. Publicar vaga + filtro discriminatório
9. Painel da empresa + candidatos
10. Marcação de presença + contestação
11. Confirmação de véspera
12. **Revisão de toda a copy contra a tabela do §2.1**

13. Fase 4: API Fastify cumprindo os contratos
14. Fase 5: Prisma, Postgres no contêiner, migrações
15. MinIO, Web Push real, Resend
16. Deploy: `docker compose up -d` por SSH no VPS, DNS do subdomínio, nginx e certbot (§9.2, §9.6)
17. Backup configurado e restauração testada
18. Depois da aprovação do sócio: domínio próprio, gateway de assinatura

Itens 1 a 12 rodam inteiramente contra o mock. Ao fim do 12 existe produto navegável e vendável sem uma linha de backend.

19. Fora do escopo

Chat interno · estrelas, notas ou comentários · qualquer processamento do pagamento do bico · verificação de antecedentes criminais · app nativo · analytics elaborado · múltiplas cidades · internacionalização.

20. Como usar com o Claude Code

O `CLAUDE.md` na raiz é a fonte das regras invioláveis e das convenções — não duplique conteúdo dele aqui. Este documento é a referência de detalhe, lida sob demanda.

Peça uma tarefa por vez, referenciando a seção:

implemente a etapa 4 do cadastro conforme §16.1, usando os tipos de §7 e a camada de §8.1